INFORME TÉCNICO FINAL

Banco Andino Colombia: base de datos OLTP verificada por quality gates

Laboratorio de Ingeniería de Bases de Datos Relacionales Aumentada con IA · Bases de Datos II · Ingeniería de Sistemas y Telecomunicaciones · Universidad de Manizales

Integrantes: Juan Diego Ortiz · Adrian Daniel Argotte · Miguel Andrés Pava · Jerónimo Soto · David Gonzáles

Fecha: 1 de octubre de 2026

Repositorio: Proyecto-BD-Banco · PostgreSQL 16 y 18.6 · Python 3.13 · todos los datos son 100 % sintéticos

1. Resumen

Diseñamos, construimos y verificamos la base de datos transaccional de un banco ficticio colombiano: clientes naturales y jurídicos, cuentas con varios titulares y ciclo de vida por eventos, transacciones con **ledger de doble partida**, usuarios internos con roles y auditoría. Seguimos el modelo del laboratorio: la IA propone, nosotros revisamos, **PostgreSQL ejecuta, las pruebas miden y el quality gate decide**. Cada gate lo cerramos solo con evidencia ejecutable.

Resultado: **los 9 gates (G0–G8) en PASS**. La base se reconstruye desde cero con un solo script (`sql/g8_reconstruir_todo.sql`, 3 a 10 minutos): 28 tablas, 10.000 clientes, 50.000 cuentas, 1.000.000 de transacciones POSTED y 2.000.000 de asientos. Termina en **G8 RECONSTRUCCIÓN = PASS**, con 0 fallas críticas.

Gate	Qué se exigía	Resultado	Evidencia
G0 Entorno	PostgreSQL, Python y repositorio listos	PASS · PostgreSQL 18.6 + Python 3.13	`g0_environment.txt`
G1 Negocio	≥ 30 reglas verificables, supuestos, riesgos	PASS · 54 reglas, 5 supuestos, 9 riesgos	`01_especificacion.md`
G2 Modelo	3FN, claves, cardinalidades, ERD	PASS · 28 tablas, 42 FK, 8 desnormalizaciones justificadas	`02_modelo_logico.md`
G3 Construcción	DDL, operaciones seguras, pruebas, concurrencia	PASS · 17 operaciones, 32 triggers, 5 roles; 49/49 pruebas + 4 de concurrencia	`03_construccion.md`, `g3_tests.txt`
G4 Datos	10K / 50K / 1M reproducibles y realistas	PASS · 48/48 métricas; mismos SHA-256 en 2 ejecuciones	`04_datos_sinteticos.md`
G5 SQL	30 consultas explicables, ≥ 5 validaciones cruzadas	PASS · 30 consultas + 7 validaciones	`g5_results.md`
G6 Calidad	Auditoría automatizada, 0 FAIL críticos, antes/después	PASS · 76 pruebas; 98,3 → 99,6; 7/7 defectos detectados	`g6_quality_before_after.md`
G7 Rendimiento	≥ 5 EXPLAIN antes/después, índices justificados, RBAC	PASS · 10 consultas; retiro × 82; RBAC 23/23	`g7_explain.md`, `g7_security.md`
G8 Cierre	Repositorio reproducible, ERD final, informe, matriz IA, defensa	PASS · reconstrucción en un paso; ERD = DDL	`g8_reconstruccion.txt`, `defensa.md`

2. Arquitectura

El modelo está en **3FN** y separado en cinco esquemas. Esa separación es la frontera de los permisos: un rol puede ver clientes y cuentas sin ver la seguridad ni la auditoría.

Esquema	Tablas	Contenido
`ref`	12	Catálogos: geografía DIVIPOLA (departamento, municipio), oficinas, monedas, tipos de documento, estados, tipos de transacción, plan de cuentas contables y **transiciones válidas de estado de una cuenta**
`core`	8	Cliente (supertipo) con persona natural y jurídica (subtipos 1:1), producto, cuenta, titularidad N:M con rol y vigencia, eventos de la cuenta y cambios de límite
`fin`	2	Transacción financiera (con clave de idempotencia y estado) y asiento contable (doble partida)
`seg`	5	Usuarios internos, roles, permisos y sus asignaciones con vigencia
`aud`	1	Bitácora de auditoría inmutable: quién (empleado y usuario de BD), qué, cuándo, imagen antes y después

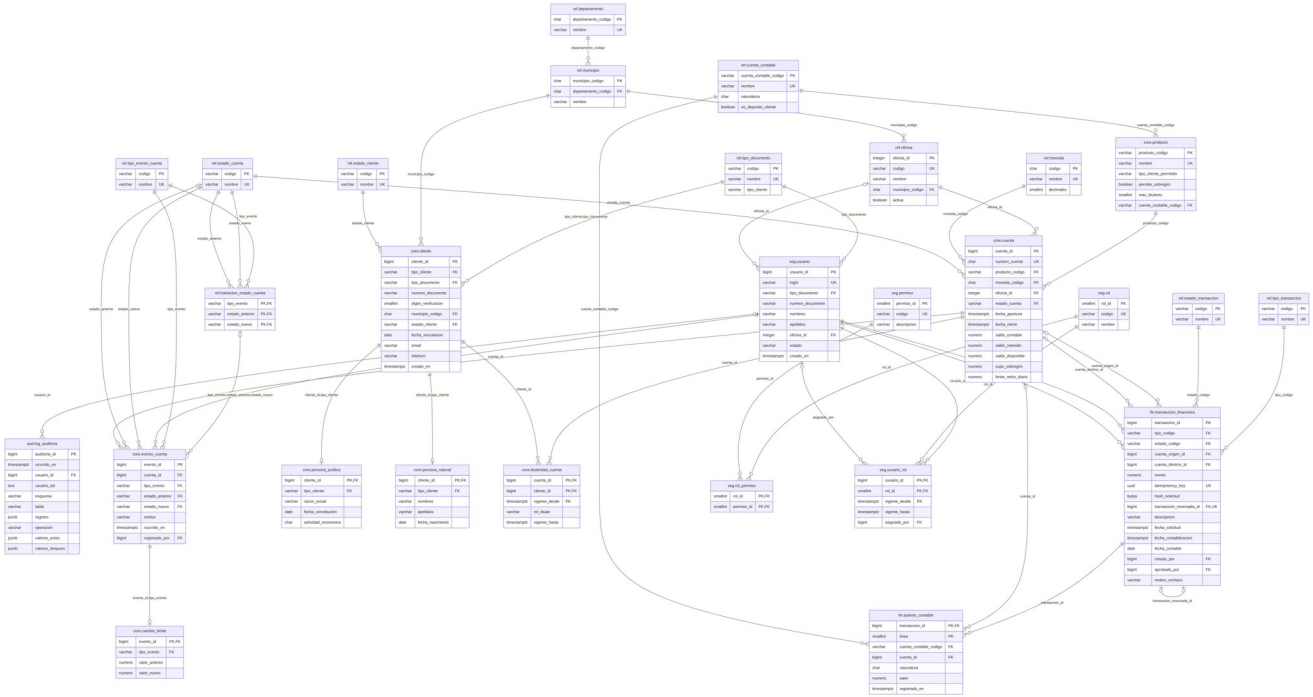


Figura 1. ERD final: 28 tablas y 42 relaciones. Se **genera desde el catálogo** de PostgreSQL (`sql/tests/g8_erd_desde_catalogo.sql`), así que coincide con el DDL ejecutado; además es idéntico al ERD aprobado en G2. Versión completa: `docs/erd_final.png`.

Cómo procesamos una operación de dinero (ejemplo: un retiro)

1. El empleado llama a `fin.fn_retirar` (función `SECURITY DEFINER` con `search_path` fijo). Ningún rol de la aplicación puede escribir directamente en las tablas de dinero.
2. `seg.fn_exigir_permiso` verifica que el **empleado** tenga el permiso por un rol vigente (segundo nivel de seguridad, error BA001).
3. Revisamos la clave de idempotencia: si la solicitud ya existe, devolvemos la misma transacción.
4. Bloqueamos la cuenta con SELECT … FOR UPDATE (en orden de cuenta_id cuando son dos) y validamos el estado, los fondos más el cupo y el **límite diario vigente**.
5. `fn_contabilizar` inserta la transacción POSTED y sus **2 asientos** en una sola sentencia, y aplica el saldo **desde los asientos**. Los triggers diferidos verifican el cuadre al confirmar y el trigger de auditoría registra quién lo hizo.

Así, los riesgos de consistencia identificados en G1 quedan controlados por la base de datos y no por la buena voluntad de la aplicación:

Riesgo (G1)	Control en PostgreSQL	Evidencia
Saldo desalineado con los asientos	Saldo aplicado desde el ledger + vista de reconciliación	G6 S01–S02 = 0; demo 3
Doble retiro simultáneo	`FOR UPDATE` en orden de `cuenta_id`, `READ COMMITTED`	Prueba de concurrencia CT-1
Reintento por falla de red	`idempotency_key UNIQUE` + `ON CONFLICT` + huella de la solicitud	CT-3; demo 4
Transferencia interrumpida	Una función = una sentencia atómica; cuadre diferido	T06; demo 5
Estado distinto del historial	El estado solo cambia con un evento; transiciones en tabla con FK	T33/T34; G6 S03
Corrección directa de transacciones	Triggers de inmutabilidad (también TRUNCATE); solo reversos	T05a–d; demo 1
Privilegios excesivos	5 roles, nada para PUBLIC, sin DML directo; permisos por empleado	RBAC 23/23 (G7)

3. Decisiones de diseño

Decisión	Alternativa descartada	Por qué	Evidencia
Ledger de doble partida como fuente de verdad; el saldo es una caché reconciliada (DM-05, DC-04)	Guardar solo el saldo	Permite reconstruir y auditar cada peso; operaciones rápidas	G6 F01 = 100 %, S01 = 0
Dinero solo por funciones seguras con bloqueo ordenado (DC-01, DC-03)	Permisos de escritura a la aplicación; `SERIALIZABLE` con reintentos	La regla no se puede saltar; sin doble retiro ni interbloqueos	CT-1 a CT-4; RBAC
Idempotencia con `uuid` único por solicitud (DM-08)	Detectar duplicados por monto y fecha	El doble clic o el reintento no crean otro movimiento (RN-38)	CT-3; demo 4
Estado de la cuenta por eventos; transiciones válidas como datos (DM-03, DM-04, DC-05)	UPDATE directo del estado validado con código	Historial completo; la base rechaza transiciones inválidas	T33, T34; G6 S03, T05
Inmutabilidad de lo contabilizado; correcciones con reverso (DM-06)	UPDATE o DELETE de la transacción	Registro contable y legal; trazabilidad completa	T05a–d, T07
Supertipo cliente con subtipos 1:1 (DM-01)	Tabla única con nulos; `INHERITS`	`INHERITS` no propaga UNIQUE ni FK; documento único (RN-01)	G2; G6 U01, R05
NUMERIC para el dinero (IA-10)	`money`, `FLOAT`	Precisión exacta, independiente de la configuración regional	G6 DS01 = 0
Índices solo con un EXPLAIN que los justifique (DC-06, G7)	Indexar todas las FK desde el inicio	Evita la sobreindexación: cada índice cuesta en escritura y espacio	6 índices; 3 casos sin índice
Carga masiva con `\copy`, triggers desactivados y validación en bloque, **registrada en la auditoría** (DC-07 + G6)	1.000.000 de INSERT con triggers	Lote 2 en 1–2 minutos; trazabilidad conservada	Controles 20/20 y 24/24; G6 A04

4. Resultados por gate

- **G3 · Construcción.** 49 pruebas en una sola transacción con ROLLBACK (39 negativas con su código de error esperado y 10 positivas), más 4 pruebas de concurrencia con dos sesiones. `run_all.sql` corre dos veces seguidas sin errores. Resultado 49/49 en PostgreSQL 16 y en nuestro 18.6.
- **G4 · Datos sintéticos.** Faker `es_CO` + un generador cronológico propio que respeta saldo, estado y límite **en cada momento**. Semilla 20260909: dos ejecuciones producen los 14 CSV con el mismo SHA-256. Las distribuciones no son uniformes: el departamento más grande tiene 9,6 veces el promedio; las cuentas por cliente tienen mediana 3 y máximo 300; el pico horario es 22,7 veces el valle.
- **G5 · SQL.** 30 consultas (10 básicas, 10 intermedias, 10 avanzadas, con ventanas, CTE y percentiles) y 7 validaciones cruzadas en PASS. Cada una tiene su explicación de negocio.
- **G6 · Calidad de datos.** 76 pruebas en las 12 categorías del Prompt 5, con puntajes por dimensión. El **antes** se midió sin corregir nada (98,3 y 1 FAIL ALTA); después de corregir, el puntaje subió a 99,6 sin FAIL. En una copia de la base se metieron 7 defectos (ledger descuadrado, saldo alterado, documento duplicado, titular huérfano…) y el gate los detectó todos: QUALITY_GATE = FAIL.
- **G7 · Rendimiento y seguridad.** Mediciones en nuestro PC (PostgreSQL 18.6):

Consulta u operación	Antes	Después	Plan después
OP1 · Retiro en caja (`fn_retirar`)	198,9 ms	**2,4 ms**	× 82
OP3 · Inactivar cuentas sin movimiento (4.018)	51,9 s	**5,4 s**	× 9,7 (índice + reescritura)
W01 · Límite diario de retiro	68,9 ms	0,010 ms	Index Scan · 16.889 → 4 buffers
W02 · Extracto mensual de una cuenta	106,9 ms	0,021 ms	Index Scan (índice parcial)
W07 · Cola de ajustes pendientes	57,9 ms	0,021 ms	Índice parcial de 16 kB
W08 · Cliente por documento	0,014 ms	0,015 ms	**No crear índice**: ya existe UNIQUE
W10 · Reporte de movimientos atípicos	3,0 s	3,5 s	**Regresión** documentada (§5)

Costo de los 6 índices: 86 MB (+18 %) y +16 s en la carga del lote 2. Seguridad: 23 pruebas con un usuario por rol, 8 accesos permitidos y 15 denegados, 0 fallas. Incluye un caso en que la base permite la operación pero la aplicación la niega porque el empleado no tiene el rol.

5. Hallazgos y errores descubiertos

Hallazgo	Cómo se detectó	Corrección	Gate
La carga masiva dejó 3,2 millones de filas sin rastro en la auditoría (triggers desactivados)	Quality gate sin corregir: A04 = FAIL	Cada carga registra usuario, hora, archivo y filas; A04 = PASS	G6
Cada retiro recorría el millón de transacciones con la cuenta bloqueada	`EXPLAIN (ANALYZE, BUFFERS)` de las funciones	Índice `(cuenta_origen_id, fecha_contable)`: retiro × 82	G7
Un índice útil para el extracto **empeoró un reporte** (el planificador estimó mal)	Comparación antes/después de las 30 consultas de G5	Se mantiene el índice; el reporte desactiva el index scan en su sesión	G7
El generador dejó 482 movimientos en cuentas por inactivar y 0 rechazos por límite	24 controles del lote 2	Tramos de inactividad corregidos; intentos que exceden el límite	G4
Comparar retiros con el límite **actual** producía días "al 180 %"	Resultado imposible en A07	Límite vigente reconstruido desde los eventos	G5
Subconsulta correlacionada de 2 min 39 s	Tiempo medido	CTE: milisegundos	G5
Vista de estado por eventos con `LATERAL`: 3 min 25 s	Prueba de G3 con el millón	`DISTINCT ON`: 4 s	G3
Tildes dañadas en Windows y filas decimales en EXPLAIN de PostgreSQL 18	Al correrlo en nuestro PC	UTF-8 forzado en los scripts; probado en PG16 y PG18	G7

Lección común: **los errores aparecieron porque se midió**. La prueba A04, el EXPLAIN y los controles de carga encontraron problemas que una revisión de código no habría visto.

6. Uso de la IA

Registramos 46 decisiones sobre recomendaciones de la IA (`docs/matriz_ia.md`) y 12 prompts con su respuesta (`prompts/prompts_y_respuestas.md`):

Aceptar	Modificar	Rechazar	Corregir (error de la IA)	Aplazar	Total
19	11	10	5	1	46

27 de 46 (59 %) no las aceptamos tal cual. Donde la IA más aportó: proponer alternativas (supertipo/subtipos, bloqueo ordenado, idempotencia), redactar SQL y documentación y revisar el modelo (Prompt 2A encontró 12 hallazgos). Donde más se equivocó: tomó decisiones sin medir (índices para los reportes, carga sin auditoría), interpretó mal instrucciones (10.000 cuentas en vez de 50.000) y no anticipó problemas de entorno (Windows, PostgreSQL 18). Ninguna de estas fallas llegó al resultado final: todas las detectó una prueba, un gate o nuestra revisión.

7. Limitaciones y mejoras

- **Supuestos S1–S5: los adoptamos nosotros sin consultarlos con el docente; los documentamos** y cada uno tiene su regla verificable.
- **Datos:** los saldos parten de 0 en septiembre de 2024 (no simulamos el histórico) y no modelamos las conversiones COP/USD.
- **Hacia 100 millones de movimientos:** particionar transacciones y asientos por mes de `fecha_contable`, índices BRIN por fecha, réplica de lectura y vistas materializadas para los reportes, índices creados después de cada carga masiva, `autovacuum` ajustado para tablas que solo reciben INSERT y un pool de conexiones. Las reglas (doble partida, inmutabilidad, idempotencia, bloqueo ordenado) se mantienen: trabajan por fila y por cuenta.
- **Operación:** correr `g8_reconstruir_todo.sql` en integración continua (GitHub Actions con PostgreSQL), monitorear con `pg_stat_statements` y usar usuarios `LOGIN` con SCRAM y SSL en lugar de los usuarios de demostración.

8. Reproducibilidad

Otro estudiante reconstruye el laboratorio completo con tres pasos, desde la raíz del repositorio:

1. `python src/generate_transactions.py`: genera el lote 2 (unos 30 s). El lote 1 ya viene en `data/lote1`.
2. `psql -U postgres -c "CREATE DATABASE banco_andino_lab"`
3. `psql -U postgres -d banco_andino_lab -f sql/g8_reconstruir_todo.sql`: modelo, 49 pruebas, cargas, perfil, quality gate y permisos. Termina en **G8 RECONSTRUCCIÓN = PASS**.

Si después Git no muestra cambios en `evidence/g4_data_profile.csv`, los datos reconstruidos son idénticos a los originales. Así lo comprobamos en nuestro PC (Windows, PostgreSQL 18.6): la reconstrucción tardó 10 minutos, terminó en PASS y el perfil salió idéntico byte a byte. El checklist final de la guía (19 puntos) lo verificamos punto por punto, con su evidencia, en `docs/defensa.md`. Ese documento también trae las respuestas a las 8 preguntas de defensa y una demostración en vivo de 5 minutos (`sql/tests/g8_demo_defensa.sql`).